code 的意思及顺序

Module 9 · P13

- Vertex AI Python client library.
  Module 6 P.143.
-

Cloud def

Cloud Computing is the on-demand delivery of computer resources via the internet with pay-for-use pricing.

SaaS / PaaS / IaaS

#salesforce #Microsoft 365

對IT控制力 ↑

SaaS 軟体, 服務由供應商管理, 付費使用

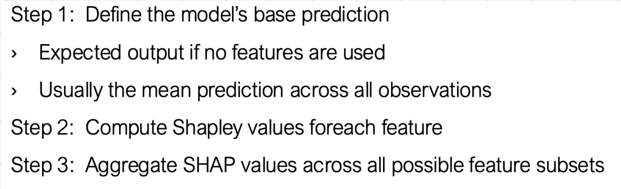
PaaS (middle ware) 軟, 硬体由 provider 提供

IaaS (hardware as a service) #AWS
硬体由 provider, data/軟体自己管

SHAP score

(+) 正影響 / (-) 負影響

Prediction result = base num + $\boxed{\text{A Feature}}$



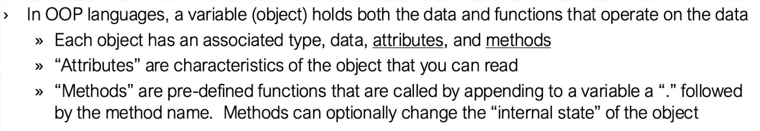
Step 1: Define the model's base prediction

- › Expected output if no features are used
- › Usually the mean prediction across all observations

Step 2: Compute Shapley values foreach feature

Step 3: Aggregate SHAP values across all possible feature subsets

Python class/object/method/ Attribute



- › In OOP languages, a variable (object) holds both the data and functions that operate on the data
  - » Each object has an associated type, data, attributes, and methods
  - » "Attributes" are characteristics of the object that you can read
  - » "Methods" are pre-defined functions that are called by appending to a variable a "." followed by the method name. Methods can optionally change the "internal state" of the object

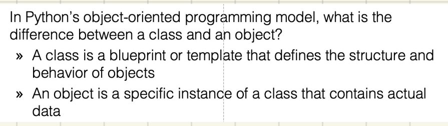
my_dataset = aiplatform. Tabulardataset (...)

↓ Object. 物件. ↓ class. 不同的模版.

Methods = model. deploy ( )
#做事 #動作.

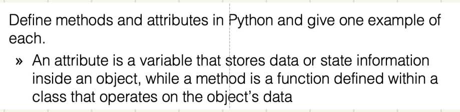
Attributes = endpoint. display_name
#查看資訊.

→ 沒有括号. ↓



In Python's object-oriented programming model, what is the difference between a class and an object?

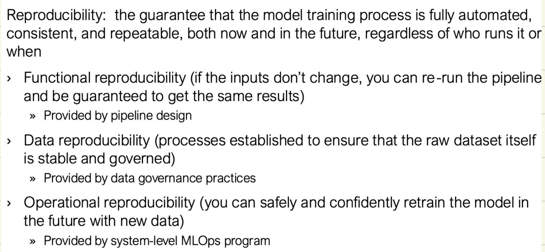
- » A class is a blueprint or template that defines the structure and behavior of objects
- » An object is a specific instance of a class that contains actual data



Define methods and attributes in Python and give one example of each.

- » An attribute is a variable that stores data or state information inside an object, while a method is a function defined within a class that operates on the object's data

Reproducibility



Reproducibility: the guarantee that the model training process is fully automated, consistent, and repeatable, both now and in the future, regardless of who runs it or when

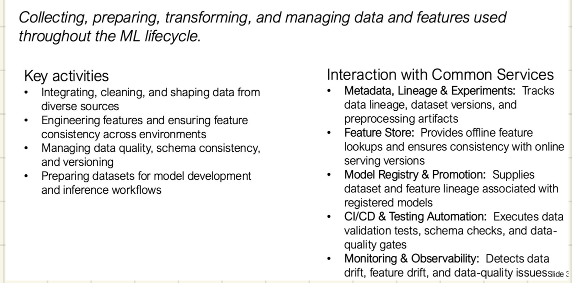
- › Functional reproducibility (if the inputs don't change, you can re-run the pipeline and be guaranteed to get the same results)
  - » Provided by pipeline design
- › Data reproducibility (processes established to ensure that the raw dataset itself is stable and governed)
  - » Provided by data governance practices
- › Operational reproducibility (you can safely and confidently retrain the model in the future with new data)
  - » Provided by system-level MLOps program

MLops def

is a board term that includes a set of infrastrature capabilities processes. and management practices that focus on the developement of data at scale

set of practices for developing deploying and managing ML systems at scale,

Data & Feature Management.



Collecting, preparing, transforming, and managing data and features used throughout the ML lifecycle.

Key activities

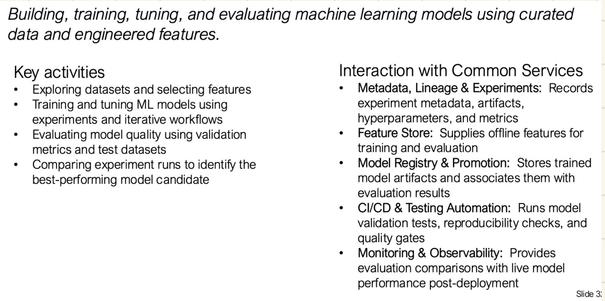
- Integrating, cleaning, and shaping data from diverse sources
- Engineering features and ensuring feature consistency across environments
- Managing data quality, schema consistency, and versioning
- Preparing datasets for model development and inference workflows

Interaction with Common Services

- **Metadata, Lineage & Experiments:** Tracks data lineage, dataset versions, and preprocessing artifacts
- **Feature Store:** Provides offline feature lookups and ensures consistency with online serving versions
- **Model Registry & Promotion:** Supplies dataset and feature lineage associated with registered models
- **CI/CD & Testing Automation:** Executes data validation tests, schema checks, and data-quality gates
- **Monitoring & Observability:** Detects data drift, feature drift, and data-quality issues

Slide 3

Model Developement & Experimentation



Building, training, tuning, and evaluating machine learning models using curated data and engineered features.

Key activities

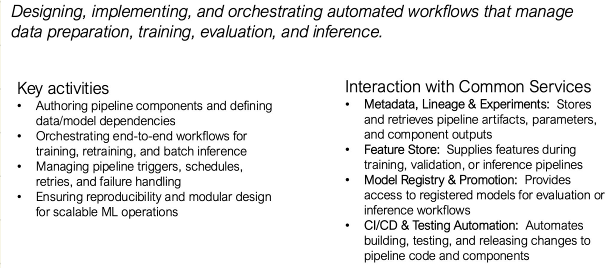
- Exploring datasets and selecting features
- Training and tuning ML models using experiments and iterative workflows
- Evaluating model quality using validation metrics and test datasets
- Comparing experiment runs to identify the best-performing model candidate

Interaction with Common Services

- **Metadata, Lineage & Experiments:** Records experiment metadata, artifacts, hyperparameters, and metrics
- **Feature Store:** Supplies offline features for training and evaluation
- **Model Registry & Promotion:** Stores trained model artifacts and associates them with evaluation results
- **CI/CD & Testing Automation:** Runs model validation tests, reproducibility checks, and quality gates
- **Monitoring & Observability:** Provides evaluation comparisons with live model performance post-deployment

Slide 3

Pipeline Engineering & Orchestration



Designing, implementing, and orchestrating automated workflows that manage data preparation, training, evaluation, and inference.

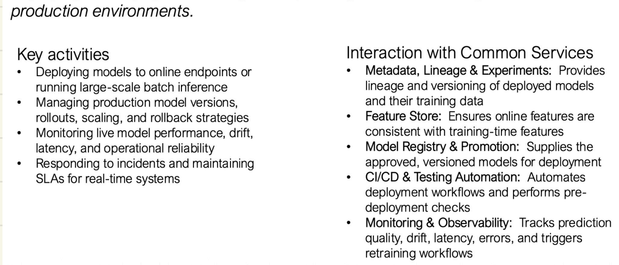
Key activities

- Authoring pipeline components and defining data/model dependencies
- Orchestrating end-to-end workflows for training, retraining, and batch inference
- Managing pipeline triggers, schedules, retries, and failure handling
- Ensuring reproducibility and modular design for scalable ML operations

Interaction with Common Services

- **Metadata, Lineage & Experiments:** Stores and retrieves pipeline artifacts, parameters, and component outputs
- **Feature Store:** Supplies features during training, validation, or inference pipelines
- **Model Registry & Promotion:** Provides access to registered models for evaluation or inference workflows
- **CI/CD & Testing Automation:** Automates building, testing, and releasing changes to pipeline code and components

Model Serving & Production Operation



Deploying, managing, monitoring, and operating machine learning models in production environments.

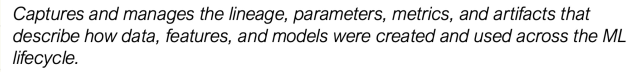
Key activities

- Deploying models to online endpoints or running large-scale batch inference
- Managing production model versions, rollouts, scaling, and rollback strategies
- Monitoring live model performance, drift, latency, and operational reliability
- Responding to incidents and maintaining SLAs for real-time systems

Interaction with Common Services

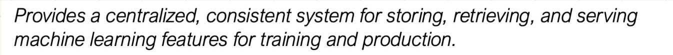
- **Metadata, Lineage & Experiments:** Provides lineage and versioning of deployed models and their training data
- **Feature Store:** Ensures online features are consistent with training-time features
- **Model Registry & Promotion:** Supplies the approved, versioned models for deployment
- **CI/CD & Testing Automation:** Automates deployment workflows and performs pre-deployment checks
- **Monitoring & Observability:** Tracks prediction quality, drift, latency, errors, and triggers retraining workflows

Metadata, leneage, experiments.



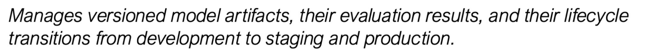
Captures and manages the lineage, parameters, metrics, and artifacts that describe how data, features, and models were created and used across the ML lifecycle.

Feature Store.



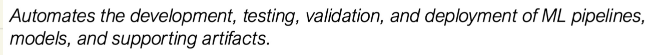
Provides a centralized, consistent system for storing, retrieving, and serving machine learning features for training and production.

Model Registry & Promotion



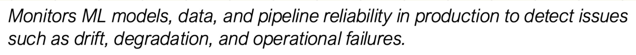
Manages versioned model artifacts, their evaluation results, and their lifecycle transitions from development to staging and production.

CI/CD Testing Automation



Automates the development, testing, validation, and deployment of ML pipelines, models, and supporting artifacts.

Monitoring &



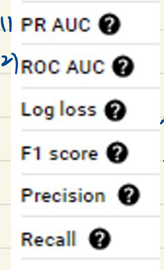
Monitors ML models, data, and pipeline reliability in production to detect issues such as drift, degradation, and operational failures.

GCP Position

- vertex AutoML：用图示化界面練model
  - # No code # GUI
- vertex Pipeline = 串聯整個workflow
  - # workflow # orchestration

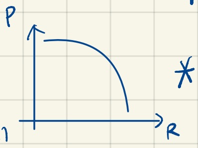
- Big Query = 大量數據運算.
  - # serveless environment # ad-hoc
- Google Cloud storage = 可存任何 type
  - # Archive # storage tiers.

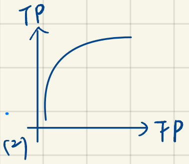
Classification Model

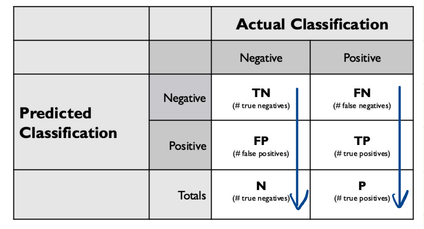


(1) PR AUC	Precision Recall AUC	右上
(2) ROC AUC	TP 和 FP 的關係	左上
Log loss	預測和真實結果距離	Low
F1 score	衡量 Precision / Recall	High
Precision	有沒有抓錯.	High
Recall	有沒有漏掉?	High.

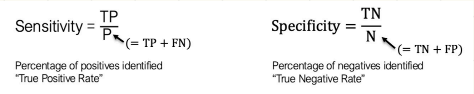
一高一低







		Actual Classification	
		Negative	Positive
Predicted Classification	Negative	**TN** (# true negatives)	**FN** (# false negatives)
	Positive	**FP** (# false positives)	**TP** (# true positives)
	Totals	**N** (# true negatives)	**P** (# true positives)

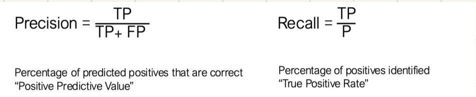


$$\text{Sensitivity} = \frac{TP}{P} \quad (= TP + FN)$$

Percentage of positives identified
"True Positive Rate"

$$\text{Specificity} = \frac{TN}{N} \quad (= TN + FP)$$

Percentage of negatives identified
"True Negative Rate"

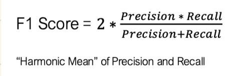


$$\text{Precision} = \frac{TP}{TP + FP}$$

Percentage of predicted positives that are correct
"Positive Predictive Value"

$$\text{Recall} = \frac{TP}{P}$$

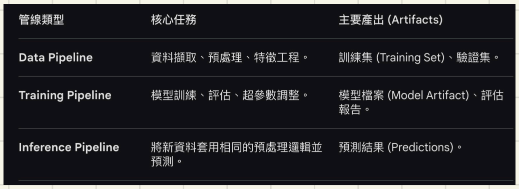
Percentage of positives identified
"True Positive Rate"



$$\text{F1 Score} = 2 * \frac{Precision * Recall}{Precision + Recall}$$

"Harmonic Mean" of Precision and Recall

Pipeline Types



管線類型	核心任務	主要產出 (Artifacts)
Data Pipeline	資料擷取、預處理、特徵工程。	訓練集 (Training Set)、驗證集。
Training Pipeline	模型訓練、評估、超參數調整。	模型檔案 (Model Artifact)、評估報告。
Inference Pipeline	將新資料套用相同的預處理邏輯並預測。	預測結果 (Predictions)。

Data Leakage

randomized → split → normalize

Def. Data leakage is unintentional Introduction of Information about the response variable that should not be made available to the training.

Leakage

- Group Leakage : 同個病人的不同次看病分到不同組
- Partition Leakage = 同時存在 train/test.
- Imputation Leakage = 只能用自己組的data 算mean. → Imputation

* Common cause: 6. Feature engineering leakage

1. Target leakage: predictors contain response
2. Hidden target
3. Future data leakage
4. Improper spliting
5. Incorrect processing

Imputation注意使用split之後的group mean.

ML Pipeline Def

Def. ML pipeline is a formalized and repeatable workflow of the best experiment

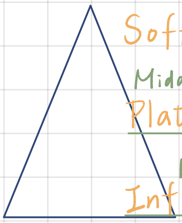
Pipelines are not for discovering models, but for running decisions you already trust. 4. Reliability

* Key goal: 1. Automation 2. Consistency 3. Scalability

* Parameters v.s. Artifacts

- △ Parameter: value / small data
- △ Artifact: reference / reproducibility & tracking

Def. Cloud computing is the on-demand delivery of computer resources via the internet with pay-for-use pricing



owned / managed by providers

Software as a service (Saas)

Middleware

foundation for building other apps

Platform as a service (Paas)

Hardware

managed by you

Infrastructure as a service (Iaas)

* Cloud Benefits

I, Cost effectiveness   II, Scalability

III. Standardization and governance

IV, Integrated modeling lifecycle pf.

* GCP Hierachy

(Organization) → (Folder) → Project

Vertex AI, BigQuery   buckets, etc

→ Managed Service → Resource

---

* Vertex AI provides:

I. Data preparation and feature engineering.

II. Model training and experiment

III. Model deployment and predictions

IV. MLOps and pipelines.

* Key interaction methods:

I. GC Console (No/Low code)

II. GC SDK CLI (Automating)

III. Programmatic libraries (Python, Java, etc.)

* Console-based AutoML

1. Vertex Managed Datasets

fast build-in Simplifies the process to train ML models

2. AutoML training

3. Model evaluation and Interpretation

Feature Importance

- SHAP (Shapley Additive Explanations)

Step 1: Define the model's base prediction

› Expected output if no features are used

› Usually the mean prediction across all observations

Step 2: Compute Shapley values foreach feature

Step 3: Aggregate SHAP values across all possible feature subsets

Classification

Focus on ✩ misclassification rate

- Confusion Metrix

		Ⓐ	
		N	P
	N	TN	FN
Ⓟ	P	FP	TP
	T	N	P

- Sensitivity $= \frac{TP}{TP + FN}$

  Specificity $= \frac{TN}{TN + FP}$

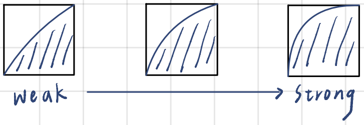
- Precision $= \frac{TP}{TP + FP}$

  Recall $= \frac{TP}{TP + FN}$

- F1-Score $= 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$

- Receiver Operating Characteristics

TP v.s. FP, C-statistic



* Python

I. Class: "blueprints" or "templates"

II. Object: specific instance of classes

III. Attribute: data stored in the object

IV. Method: Functions tied to the object

* Explanation

I. Feature attributions: SHAP, LIME

II. Global explanations

Def. MLOps is a board term that includes a set of infrastructure capabilities, processes, and management practices that focus on the development of data at scale

Set of practices for developing, deploying and managing ML systems at scale.

* Core Objectives

I. Automation   II. Collaboration

III. Reproducibility   IV. Quality & Governance

Def. Reproducibility is the guarantee that the training process is automated, consistent and repeatable. Functional, Data, Operational

Def. ML pipeline is a formalized and repeatable workflow of the best experiment

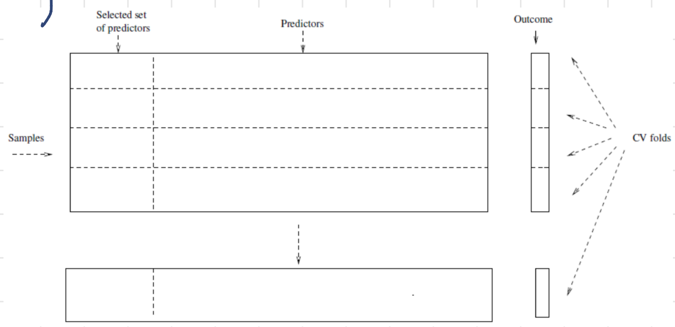
Pipelines are not for discovering models, but for running decisions you already trust. 4. Reliability

* Key goal: 1. Automation 2. Consistency 3. Scalability

Def. Data leakage is unintentional Introduction of Information about the response variable that should not be made available to the training.

* Common cause: 6. Feature engineering leakage
  1. Target leakage: predictors contain response
  2. Hidden target 3. Future data leakage
  4. Improper spliting 5. Incorrect processing

Right



Everything must be learned only from the training data and then to val.

Def. Orchestrated pipelines are automated workflows where multiple ind. task coordinated

* Enable scalability, portability, reproducibility

* Each component runs in its own container

* Containers, Orchestration Tools

* Kubeflow: open-source ML platform (Vextex)
  1. Handle execution, scheduling, dep. management
  2. Pipelines = workflows, components = steps

* Parameters v.s. Artifacts
  ▵ Parameter: value / small data
  ▵ Artifact: reference / reproducibility & tracking

* Pipelines:
  1. Training: prep. → training → eval.
  2. Inference (prediction) pipeline:
    ▵ must match training preprocessing
    ▵ Input → Apply same feature transf. → predict → post-process → Output

---

* Core Assets:
  I. Data Asset: raw and processed DS.
  II. Feature Asset: Engineered var. for pred.
  III. Model Asset: Trained ML models

* ML Systems natually accumulate hidden technical debt due to complexity

* First-class Asset: Secure, Governed, Named, Discoverable, Reusable, Versioned, Traceable

* Opetional Assets:
  Pipelines, Experiments, Endpoints, Training Jobs.

Python SDK

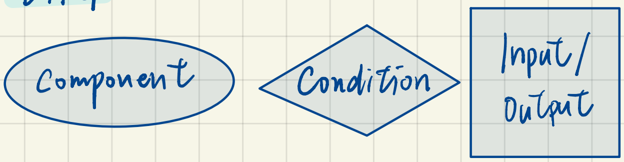
```
@component1(packages to install=[" "])
def __c1__(
    input_name_1 = str,
    output_name_1 = Output[dataset],
)
@component2
def __c2__(
    input_name_2 = Input[dataset]
    output_name_2 = Output[model ],

)
@pipeline(
    name =
)
def ______(
    input = str,
    # no output
):
    task_1 = c1(
      input_name_1 = input,
    )
    task_2 = c2(
      input_name_2 = task_1.outputs[output_name_1]
```

import ____

init ____

Tabular Job. ____

DAG


Component
Condition
Input/
Output

4/23 573 mid 2

final Project

準備 530

5/12 530 final

① 5月玩

6月回台

7、8月 project.

② 全部待在台灣.

8/23 開學.

產業 = ① supply chain

② Business

內容 = 不想只做 Tableau.

如果可以跟碰 GCP / Prediction / Model